

Stato	Completato
Iniziato	mercoledì, 21 gennaio 2026, 15:42
Terminato	mercoledì, 21 gennaio 2026, 17:12
Tempo impiegato	1 ora 29 min.
Valutazione	8,65 su un massimo di 17,50 (49,43%)
Feedback	Thanks for your attempt, all the best!

Domanda **1**

Parzialmente corretta

Punteggio ottenuto 0,88 su 1,00

We have seen a lot of tricks and tweaks for training neural networks, but, at the end of the day, they all try to either increase generalization or training speed/stability. For each of the following techniques, indicate which of the two goals they primarily address.

Generalization	Training speed/stability		
☒	☐	Early stopping	☒
☒	☐	Weight decay	☒
☐	☒	Learning rate	☒
☐	☒	Shortcut / skip connection	☒
☐	☒	Xavier initialization	☐
☐	☒	ReLU activation functions	☒
☐	☒	Batch Normalization	☒
☒	☐	Dropout	☒

Early stopping: Generalization

Weight decay: Generalization

Learning rate: Training speed/stability

Shortcut / skip connection: Training speed/stability

Xavier initialization: Training speed/stability

ReLU activation functions: Training speed/stability

Batch Normalization: Training speed/stability

Dropout: Generalization

Domanda 2

Parzialmente corretta

Punteggio ottenuto 0,83 su 2,00

The learning rate is a critical hyperparameter of gradient descent because ...

- ☒ a. It might lead to unstable training if set too high.  It might oscillate around the minimum or in steep minima valleys.
- ☐ b. It must be modified necessarily monotonically during training, so the starting value is critical if not exactly selected as it cannot be reversed.
- ☐ c. If set low, it guarantees the training is stable and will always converge.
- ☒ d. If you set it high the gradient descent moves faster  Correct the higher the learning rate, the faster the gradient motion, although this might be unstable if too much.
- ☐ e. It can reduce the effectiveness of neural networks based on ReLU activation functions if set too low.
- ☒ f. If it is modified during training, starting low and increasing, it  It is the other way round, you should start with a relatively high learning rate and then reduce it progressively at the end of the training.
- ☒ g. It allows, if set low, to finetune pre-trained models keeping their "knowledge"  Yes, it should be around 1 tenth of the original learning rate.

Your answer is partially correct.

Hai selezionato correttamente 3.

Le risposte corrette sono: It might lead to unstable training if set too high., If you set it high the gradient descent moves faster toward the minimum of the loss function., If set low, it guarantees the training is stable and will always converge., It allows, if set low, to finetune pre-trained models keeping their "knowledge"

Domanda 3

Parzialmente corretta

Punteggio ottenuto 1,60 su 2,00

With reference to the word2vec model to perform word embedding select which of the following is correct.

True	False		
☐	☒	Word embeddings in word2vec represent discrete words and as such they cannot be added nor subtracted.	☒ As they represent in a distributed way the context of the targeted word (Continuous Bag of Words) they can be summed and added, obtaining the unexpected zero shot learning results shown in class.
☐	☒	Word embedding is considered to be trained in an unsupervised way because it does not minimize a learning loss, but it optimizes vector distances in the embedding space.	☒ No, word2vec for instance, minimizes cross-entropy as it is a classifier over all possible words.
☐	☒	Word2vec was a major improvement because, unlike one-hot-encoding, it is sparse and, because of this, it represents words with fewer parameters.	☒ word2vec is not sparse; it tries to make the embedding denser and smoother via densification. The embedding space is smaller because we make a projection not because of sparser.
☐	☒	Word2vec is not trained in an unsupervised manner.	☒ False
☒	☐	Word2Vec is considered to be trained in an unsupervised way because it implements a classical neural autoencoder architecture.	☒ No, it does not implement a classical neural autoencoder, as the input is different from the output and it is not learned via reconstruction error.

Word embeddings in word2vec represent discrete words and as such they cannot be added nor subtracted.: False

Word embedding is considered to be trained in an unsupervised way because it does not minimize a learning loss, but it optimizes vector distances in the embedding space.: False

Word2vec was a major improvement because, unlike one-hot-encoding, it is sparse and, because of this, it represents words with fewer parameters.: False

Word2vec is not trained in an unsupervised manner.: False

Word2Vec is considered to be trained in an unsupervised way because it implements a classical neural autoencoder architecture.: False

Domanda 4

Parzialmente corretta

Punteggio ottenuto 1,60 su 2,00

In the following cases, select if the design choice is correct or not according to the description and the assumption.

Note: With "correct" we mean both the model and the assumptions make sense, not the fact that you can hack pytorch to implement it ...

Correct	Not correct		
☒	☐	We want to build a multi-set encoder , i.e., a model to generate the encoding of an unordered list of items where items can repeat. Items are represented via a one-hot encoding. Each multi-set of items, which might have a generic finite cardinality, is given as input to an LSTM (assume zero initialization), one item at a time, after sorting them. At the end of the items, the LSTM will encode the multi-set in its state. The same multi-set, provided the initialization, will be always encoded the same way.	☒ Yes, sorting the elements makes the output of the embedding permutation invariant.
☒	☐	We could implement a simple spellchecker using a bidirectional LSTM to have a look-ahead effect and avoid the representation bottleneck.	☒ Yes, just use a bidirectional LSTM with a sigmoidal output and train it as a classifier to predict if a certain word is properly spelled or not.
☒	☐	We want to develop a sentiment analysis model able to predict the mood of a sentence. We are undecided between a 1-layer Bidirectional LSTM and a 2-layers hierachical/stacked LSTM. Thus, we go for a 2 layer hierachical/stacked LSTM made by 2 Bidirectional LSTM layers one after the other, followed by a sigmoidal layer.	☐ Yes it is a legitimate design for this problem.
☐	☒	We want to build a set encoder , i.e., a model to generate the encoding of an unordered list of items. Items are represented via a one-hot encoding. Each set of items, which may have a generic finite cardinality, is given as input to an LSTM (assuming zero initialization), one item at a time. At the end of the items, the set will be encoded in the LSTM state. Because the LSTM is permutation invariant, the same set will always be encoded the same way.	☒ It is not correct because the LSTM are not permutation invariant, they are the opposite as their state depends on the order of the input.
☐	☒	We want to reverse a sentence (starting from the last word to the first) and use a sequential model for this. We cannot use an LSTM, as in the generation of the new sentence, it has to progress forward in an autoregressive way. We can only resort to a bidirectional LSTM.	☒ There is no reason we cannot use an LSTM, it might not be as effective as other architectures, but the fact it is autoregressive has no implication in this case.

We want to build a **multi-set encoder**, i.e., a model to generate the encoding of an unordered list of items where items can repeat. Items are represented via a one-hot encoding. Each multi-set of items, which might have a generic finite cardinality, is given as input to an LSTM (assume zero initialization), one item at a time, after sorting them. At the end of the items, the LSTM will encode the multi-set in its state. The same multi-set, provided the initialization, will be always encoded the same way.: Correct

We could implement a simple spellchecker using a bidirectional LSTM to have a look-ahead effect and avoid the representation bottleneck.: Correct

We want to develop a sentiment analysis model able to predict the mood of a sentence. We are undecided between a 1-layer Bidirectional LSTM and a 2-layers hierachical/stacked LSTM. Thus, we go for a 2 layer hierachical/stacked LSTM made by 2 Bidirectional LSTM layers one after the other, followed by a sigmoidal layer.: Correct

We want to build a **set encoder**, i.e., a model to generate the encoding of an unordered list of items. Items are represented via a one-hot encoding. Each set of items, which may have a generic finite cardinality, is given as input to an LSTM (assuming zero

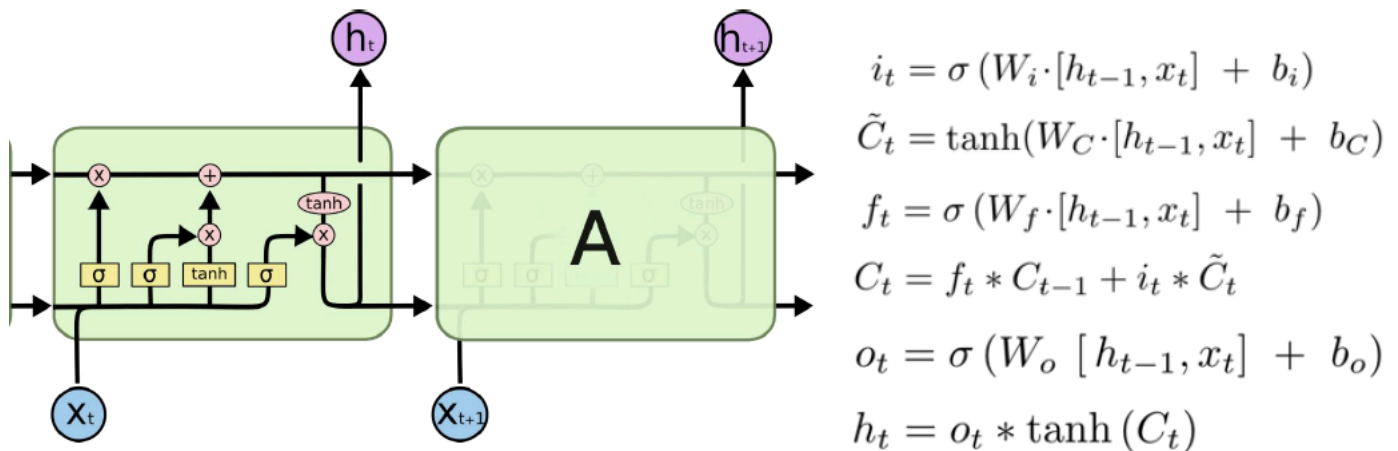
initialization), one item at a time. At the end of the items, the set will be encoded in the LSTM state. Because the LSTM is permutation invariant, the same set will always be encoded the same way.: Not correct

We want to reverse a sentence (starting from the last word to the first) and use a sequential model for this. We cannot use an LSTM, as in the generation of the new sentence, it has to progress forward in an autoregressive way. We can only resort to a bidirectional LSTM.: Not correct

Domanda 5

Completo

Non valutata



Consider the LSTM model in the picture with its equations. Answer each of the following questions using a single sentence.

0) What does LSTM stand for?

1-6) What is the meaning/semantics, in terms of LSTM functioning, of each of the 6 equations above (numbered from 1 to 6 top to bottom)?

Note: not using the provided template or not answering with a single sentence penalizes the exercise evaluation.

0) <LSTM stands for Long Short Term Memory>

1) <It corresponds to the sigmoid function on the input in the center of the image, the sigmoid to the left of the yellow box tanh>

2) <It is the tanh yellow box function which evaluates the input in the center of the image>

3) <It corresponds to the memory cell in LSTM, the sigmoid function with a (x) all on the left of the picture>

4) <It is the + part in the scheme above, f_t comes from the previous step. the two previous evaluated contributions are summed here>

5) <It is the contribution to the output evaluated with the sigmoid function at the right of the yellow box tanh>

6) <It is the final output at time t, it is the block sigmoid->(x)-(tanh)- in the right part of the picture>

Domanda 6

Parzialmente corretta

Punteggio ottenuto 0,63 su 2,00

Mark all the answers that are true.

When answering a question: Remember that our goal is to assess your understanding of the course materials, rather than your programming skills in Python. Our questions are not about what is "technically possible in Python", but rather about the fundamentals of the deep learning models and what are the best practices.

act as a Neural Network expert, as you are expected to be, not as a Python programming expert.

Wrong answers will result in negative points.

Scegli una o più alternative:

- ☐ a. Batch normalization layers standardize the activations of each channel of the input, by subtracting the sample mean and dividing by the sample standard deviation. Thus, it has no parameters as the operation is well defined for each channel.
- ☒ b. The convolutional backbone of a CNN can be seen as a learnable nonlinear filter scanning the entire image ✔ True
- ☒ c. Convolution and Correlation in image processing are translation invariant operations (ignore image boundaries and padding) ✔ True, look at the definition
- ☒ d. Deep learning revolution consists in the fact that it became computationally possible to train neural networks with many layers. ✘ False, Deep Learning consists in learning data-driven features rather than relying on experts to compute features.
- ☐ e. Expert driven features are very portable to classification tasks in images but not to regression tasks
- ☒ f. Regression, Classification and Anomaly Detection are supervised learning tasks. ✘ False, Anomaly/Outlier detection is well known to be an unsupervised learning task
- ☐ g. A Conv2D layer requires the number of channels in the input to correspond to the spatial size of the filter.
- ☐ h. Convolution and Correlation in image processing are defined as operations that, given an image in the inputs, computes a cascade of multiple local and linear transformations, each followed by a ReLU activation.
- ☐ i. Training an ensemble of CNN by changing the weight initialization is a good strategy to reduce classification errors.
- ☒ j. In binary classification it is possible to have more than two classes, as long as the last layer of the network has more than two output neurons. ✘ False, binary classification is defined as the problem of classifying inputs in two classes.
- ☐ k. Correlation can be seen as a very rudimental solution for of template matching.
- ☒ l. A trained linear classifier applied to an image corresponds to computing correlation against a set of learned templates, one per each class. The template returning the largest correlation value against the input image defines the output class, up to bias correction. ✔ True, as illustrated in the slides

Your answer is partially correct.

Hai selezionato troppe opzioni.

Le risposte corrette sono: Convolution and Correlation in image processing are translation invariant operations (ignore image boundaries and padding), Correlation can be seen as a very rudimental solution for of template matching., A trained linear classifier applied to an image corresponds to computing correlation against a set of learned templates, one per each class. The template returning the largest correlation value against the input image defines the output class, up to bias correction., The convolutional backbone of a CNN can be seen as a learnable nonlinear filter scanning the entire image, Training an ensemble of CNN by changing the weight initialization is a good strategy to reduce classification errors.

Domanda 7

Parzialmente corretta

Punteggio ottenuto 0,83 su 2,00

Mark all the answers that are true.

When answering a question: Remember that our goal is to assess your understanding of the course materials, rather than your programming skills in Python. Our questions are not about what is "technically possible in Python", but rather about the fundamentals of the deep learning models and what are the best practices.

act as a Neural Network expert, as you are expected to be, not as a Python programming expert.

Wrong answers will result in negative points.

Scegli una o più alternative:

- ☒ a. Convolutional layers can be configured to reduce the spatial extent of the input volume ✔ True, there is the stride option for this purpose
- ☐ b. Convolutional backbones leveraging skip connections (like the ResNet50) are not good candidates for performing fine tuning since the skip connections can prevent weights update.
- ☐ c. The number of operations computed in a Conv2D layer is comparable to the number of operations performed by a dense layer taking as input a vector having as many neurons as the number of parameters in the filters.
- ☐ d. The conv2D layers in the LeNet architectures use padding option "same" to preserve the spatial extent of the activations, which is important to have a compact feature vector to be classified by a small MLP
- ☐ e. In a trained CNN, each convolutional filter contributes to the computation of the values inside every neuron in the output volume.
- ☒ f. Inference by the LeNet corresponds to computing correlation against multiple templates learned during training. ✘ FALSE: correlation would be linear, LeNet is not. Even referring to the convolutional block, the LeNet architecture also includes nonlinearities and pooling.
- ☒ g. It is good to perform preprocessing when training a CNN. One of the most standard pre-processing options consists in subtracting the average image computed on the entire dataset, before performing cross-validation. ✘ FALSE: preprocessing parameters need to be computed on each training fold, not from the entire dataset
- ☒ h. A conv2D layer without nonlinear activation functions and taking as input a volume of size $N \times N \times C$, corresponds to a linear operation that computes the product of the vectorized input against a very large sparse matrix, where nonzero entries are along multiple diagonal lines. Weights are replicated every $N \times N$ rows of the matrix. The bias vector is constant for many rows. ✔ True, look how the conv2D layer can be written in matrix form
- ☐ i. Convolutional backbones stacking multiple conv2D layers, all having linear activations, are equivalent to a single conv2D layer having filters of an appropriate size and in a number that corresponds to the number of output channels of the backbone.
- ☐ j. ReLU is a nonlinear activation function, as it performs thresholding. Leaky ReLU instead (i.e., a layer that rescales linearly by a constant < 1 all the negative inputs) is a linear function, therefore this should not be used in CNN activations with multiple layers.
- ☐ k. A conv2D layer (without nonlinear activation functions and with padding) having K filters that takes as input a volume of size $N \times N \times C$, corresponds to the product against the vectorized input and a sparse matrix of $N \times N \times C$ columns. The bias vector is also $N \times N \times C$ and has constant terms every $N \times N$ rows.

Your answer is partially correct.

Hai selezionato troppe opzioni.

Le risposte corrette sono: Convolutional backbones stacking multiple conv2D layers, all having linear activations, are equivalent to a single conv2D layer having filters of an appropriate size and in a number that corresponds to the number of output channels of the backbone., Convolutional layers can be configured to reduce the spatial extent of the input volume, A conv2D layer without nonlinear activation functions and taking as input a volume of size $N \times N \times C$, corresponds to a linear operation that computes the product of the vectorized input against a very large sparse matrix, where nonzero entries are along multiple diagonal lines. Weights are replicated every $N \times N$ rows of the matrix. The bias vector is constant for many rows.

Domanda 8

Risposta errata

Punteggio ottenuto 0,00 su 2,00

Mark all the answers that are true.

When answering a question: Remember that our goal is to assess your understanding of the course materials, rather than your programming skills in Python. Our questions are not about what is "technically possible in Python", but rather about the fundamentals of the deep learning models and what are the best practices.

act as a Neural Network expert, as you are expected to be, not as a Python programming expert.

Wrong answers will result in negative points.

Scegli una o più alternative:

- ☐ a. The R-CNN architecture leverages a non-trainable region proposal algorithm to identify potential locations where objects are. Then, regions are cropped, warped and classified.
- ☒ b. The RPN computes a set of probabilities, denoted as objectness scores, that are used to select the most likely locations where objects are. ✔ TRUE: no class information is considered in the RPN
- ☐ c. The Mask R-CNN performs instance segmentation by invoking an independent U-Net on each bounding box detected by the Faster R-CNN architecture.
- ☒ d. The Fast R-CNN architecture is a more efficient implementation of the previous R-CNN, where computations are sped up by using separable convolutions, as in MobileNet. ✘ FALSE: the speed up is due to mapping the proposals from the RPA to the latent space and then using a ROI pooling layer to get a fixed-size vector
- ☐ e. The simplest solution to perform object detection consists in feeding multiple times the same test image to a classification and localization network, and in accumulating detected objects.
- ☐ f. YOLO is a single-shot network, which directly estimates multiple bounding boxes and object classes from the input image, without resorting to a separate stage where regions proposals are identified.
- ☒ g. When fine tuning an object detection network, it is necessary to modify the anchors in the RPN to take into account the number of object classes to be detected. ✘ FALSE: Anchors are used in the RPN and are class-agnostic
- ☒ h. Anchors are used in the RPN to provide a set of region proposals, which will be further refined by the detection head. ✔ TRUE
- ☒ i. CNNs performing both localization and classification need to address a multi-task learning problem, where MSE and cross-entropy loss are combined in a weighted sum. ✔ TRUE!
- ☒ j. The Faster R-CNN architecture was trained using a standard end-to-end optimization procedure, similarly to a localization and classification network. ✘ FALSE: it requires separate training stage for the RPN, the detection head and the backbone. The training loop was illustrated in the paper and reported in the slides.

Your answer is incorrect.

Le risposte corrette sono: CNNs performing both localization and classification need to address a multi-task learning problem, where MSE and cross-entropy loss are combined in a weighted sum., YOLO is a single-shot network, which directly estimates multiple bounding boxes and object classes from the input image, without resorting to a separate stage where regions proposals are identified., Anchors are used in the RPN to provide a set of region proposals, which will be further refined by the detection head., The RPN computes a set of probabilities, denoted as objectness scores, that are used to select the most likely locations where objects are., The R-CNN architecture leverages a non-trainable region proposal algorithm to identify potential locations where objects are. Then, regions are cropped, warped and classified.

Domanda 9

Risposta non data

Non valutata

Consider the following code

```
# Import PyTorch
import torch
from torch import nn
from torch.nn import functional as F
from torchsummary import summary

BATCH_SIZE = 64
INPUT_SIZE = (256, 256) # (Height, Width)
NUM_CLASSES = 3
CRITERION = nn.MSELoss()

class Model(nn.Module):
    def __init__(self, in_channels=3, num_classes=8):
        super().__init__()
        self.c0 = nn.Conv2d(in_channels=in_channels,
                             out_channels=32,
                             kernel_size=3,
                             padding=1)
        self.r0 = nn.ReLU(inplace=True)
        self.p1 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.c1 = nn.Conv2d(in_channels=32,
                             out_channels=64,
                             kernel_size=3,
                             padding=1)
        self.r1 = nn.ReLU(inplace=True)
        self.p2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.c2 = nn.Conv2d(in_channels=64,
                             out_channels=128,
                             kernel_size=3,
                             padding=1)
        self.r2 = nn.ReLU(inplace=True)
        self.d = nn.Dropout(0.2)

        self.u1 = nn.Upsample(scale_factor=2, mode='nearest')
        self.c3 = nn.Conv2d(in_channels=192,
                             out_channels=64,
                             kernel_size=3,
                             padding=1)
        self.r3 = nn.ReLU(inplace=True)

        self.u2 = nn.Upsample(scale_factor=2, mode='nearest')
        self.c4 = nn.Conv2d(in_channels=96,
                             out_channels=32,
                             kernel_size=3,
                             padding=1)
        self.r4 = nn.ReLU(inplace=True)

        self.c5 = nn.Conv2d(in_channels=32,
                             out_channels=16,
                             kernel_size=1,
                             padding=0)
        self.r5 = nn.ReLU(inplace=True)
```

```
self.c6 = nn.Conv2d(in_channels=16,
                    out_channels=8,
                    kernel_size=1,
                    padding=0)
self.r6 = nn.ReLU(inplace=True)

self.output = nn.Conv2d(8, num_classes, kernel_size=1)

def forward(self, x):
    c0 = self.c0(x)
    r0 = self.r0(c0)
    p1 = self.p1(r0)

    c1 = self.c1(p1)
    r1 = self.r1(c1)
    p2 = self.p2(r1)

    c2 = self.c2(p2)
    r2 = self.r2(c2)
    r2 = self.d(r2)

    u1 = self.u1(r2)
    u1 = torch.cat([u1, r1], dim=1)
    c3 = self.c3(u1)
    r3 = self.r3(c3)

    u2 = self.u2(r3)
    u2 = torch.cat([u2, r0], dim=1)
    c4 = self.c4(u2)
    r4 = self.r4(c4)

    c5 = self.c5(r4)
    r5 = self.r5(c5)
    c6 = self.c6(r5)
    r6 = self.r6(c6)

    output = self.output(r6)

    return output

# Create model and move to device
model = Model(in_channels=3, num_classes=NUM_CLASSES).to(device)

# Display model summary
print("Model Summary:")
summary(model, input_size=(3, INPUT_SIZE[0], INPUT_SIZE[1]))

# Inference command
pred = model(image).cpu().numpy()
```


Domanda 10

Parzialmente corretta

Punteggio ottenuto 2,28 su 2,50

Remember that this question will be automatically rated and that the code has been written in Pytorch. Therefore, you have to consider the following details very carefully:

- activations dimensions are stored in a tree vector component C x H x W
- when the activations become a vector, these are stacked along the first dimension (the channel, C). Therefore, if your latent representation becomes a 3 element vector, enter as sizes C:3 H:1 W:1.
- enter digits only and do not use "." or "," or "-" or "[]" or "[]". Say write "1234" and not "1,234" or "1.234". Write "0", not "-" or "[]".
- do not enter formulas, just resulting numbers

All the numbers have to be entered here, the text in the previous question is just meant for you to design the solution.

Layer (type)	Output Shape	Param #
Conv2d-1	C: 32 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 896 ✓
ReLU-2	C: 32 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 0 ✓
MaxPool2d-3	C: 32 ✓ , H: 128 ✓ , W: 128 ✓	Params #: 0 ✓
Conv2d-4	C: 64 ✓ , H: 128 ✓ , W: 128 ✓	Params #: 18496 ✓
ReLU-5	C: 64 ✓ , H: 128 ✓ , W: 128 ✓	Params #: 0 ✓
MaxPool2d-6	C: 64 ✓ , H: 64 ✓ , W: 64 ✓	Params #: 0 ✓
Conv2d-7	C: 128 ✓ , H: 64 ✓ , W: 64 ✓	Params #: 73856 ✓
ReLU-8	C: 128 ✓ , H: 64 ✓ , W: 64 ✓	Params #: 0 ✓
Dropout-9	C: 128 ✓ , H: 64 ✓ , W: 64 ✓	Params #: 0 ✓
Upsample-10	C: 192 ✗ , H: 128 ✓ , W: 128 ✓	Params #: 0 ✓
Conv2d-11	C: 64 ✓ , H: 128 ✓ , W: 128 ✓	Params #: 110656 ✓
ReLU-12	C: 64 ✓ , H: 128 ✓ , W: 128 ✓	Params #: 0 ✓
Upsample-13	C: 96 ✗ , H: 256 ✓ , W: 256 ✓	Params #: 0 ✓
Conv2d-14	C: 32 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 27680 ✓
ReLU-15	C: 32 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 0 ✓
Conv2d-16	C: 16 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 528 ✓
ReLU-17	C: 16 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 0 ✓
Conv2d-18	C: 8 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 136 ✓
ReLU-19	C: 8 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 0 ✓
Conv2d-20	C: 3 ✓ , H: 256 ✓ , W: 256 ✓	Params #: 27 ✓
=====		
Total params:	232275 ✓	
Trainable params:	232275 ✓	
Non-trainable params:	0 ✓	

Model Summary:

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 32, 256, 256]	896
ReLU-2	[-1, 32, 256, 256]	0
MaxPool2d-3	[-1, 32, 128, 128]	0
Conv2d-4	[-1, 64, 128, 128]	18,496
ReLU-5	[-1, 64, 128, 128]	0
MaxPool2d-6	[-1, 64, 64, 64]	0
Conv2d-7	[-1, 128, 64, 64]	73,856
ReLU-8	[-1, 128, 64, 64]	0
Dropout-9	[-1, 128, 64, 64]	0
Upsample-10	[-1, 128, 128, 128]	0
Conv2d-11	[-1, 64, 128, 128]	110,656
ReLU-12	[-1, 64, 128, 128]	0
Upsample-13	[-1, 64, 256, 256]	0
Conv2d-14	[-1, 32, 256, 256]	27,680
ReLU-15	[-1, 32, 256, 256]	0
Conv2d-16	[-1, 16, 256, 256]	528
ReLU-17	[-1, 16, 256, 256]	0
Conv2d-18	[-1, 8, 256, 256]	136
ReLU-19	[-1, 8, 256, 256]	0
Conv2d-20	[-1, 3, 256, 256]	27

Total params: 232275

Trainable params: 232275

Non-trainable params: 0

Domanda 11

Risposta errata

Punteggio ottenuto 0,00 su 2,00

Mark all the tasks that the above network can be trained for.

Please, refrain from marking answers that are "technically and eventually possible in Python", and stick to those tasks that a Neural Network expert (as you are expected to be) would use this network for.

Needless to say, assume you are always provided with the required training data for these tasks, while you cannot change the architecture and training options for the above model.

Scegli una o più alternative:

- ☐ a. This is a segmentation network that segments an input image into person, cat and background classes
- ☒ b. Given a picture of a building, the network estimates its height, width and depth ✗ This is a UNet-like regression network, look at the activations and the output size. Output size is not compliant with the task
- ☒ c. This network performs colorization: it takes as input a picture of an handwritten text (similar to MNIST images but containing a few words) and it adds colors (colouring correct words in blue and typos in red) ✗ The number of channels of the network is not compliant with a grayscale image. grayscale images (and the MNIST images we have seen in the course) have a single channel.
- ☐ d. Given as input a photo of a checkerboard where chess are set, the network modifies the image to highlight which item to move and where by drawing circles.
- ☐ e. This is a network that performs the artistic effect in portrait images, such as the subjects is in color, while the background is in different tones of beige
- ☐ f. This network takes as input an image and output 1 if the image is surely AI-generated, 0 if it is surely true and a number between 0,1 when the the network is not absolutely confident on that
- ☐ g. Given an aerial image, the network predicts in each pixel the total amount of rainfalls, of snow and of UV radiation received over the year.
- ☐ h. This network takes as input a google map picture of a city, and estimates thee pollution parameters in each pixel: bioxide, monoxide and CO2 emissions
- ☐ i. The network takes as input an X-ray image and in each pixel predicts the probability of having a bone, a body tissue or background.
- ☒ j. This network classifies images containing an animal and tells whether this is a Horse, a Shark, or a Bird ✗ This is a UNet-like regression network, look at the activations and the output size
- ☒ k. This is a network that suppresses noise in natural images ✓ Correct, this is a regression problem over three output values per pixel

Your answer is incorrect.

Le risposte corrette sono: Given an aerial image, the network predicts in each pixel the total amount of rainfalls, of snow and of UV radiation received over the year., This is a network that suppresses noise in natural images, This is a network that performs the artistic effect in portrait images, such as the subjects is in color, while the background is in different tones of beige, This network takes as input a google map picture of a city, and estimates thee pollution parameters in each pixel: bioxide, monoxide and CO2 emissions, Given as input a photo of a checkerboard where chess are set, the network modifies the image to highlight which item to move and where by drawing circles.